

SEMINARSKI RAD

Implementacija i simulacija Basic HotStuff BFT algoritma

Projekat iz predmeta: Primenjeni algoritmi u upravljačkim
sistemima

Student: Ognjen Perić
Implementacija: Go, HTML/JavaScript i Mermaid

Novi Sad, 2026.

Sažetak

Ovaj rad opisuje implementaciju obrazovnog simulatora algoritma Basic HotStuff za postizanje konsenzusa u prisustvu vizantijskih grešaka. Simulator je napisan u programskom jeziku Go i modeluje četiri replike, od kojih jedna može biti neispravna ili zlonamerna. Implementirani tok prati neulančanu varijantu algoritma iz rada o HotStuff-u i sastoji se od faza **NEW-VIEW**, **PREPARE**, **PRE-COMMIT**, **COMMIT** i **DECIDE**. Replike razmenjuju autentifikovane poruke preko simulirane mreže, glasove potpisuju Ed25519 ključevima i formiraju sertifikate kvoruma. Kao konkretna primena korišćena je deterministička bankarska mašina stanja, dok veb interfejs u realnom vremenu prikazuje sekvencu protokolskih poruka i tekstualni dnevnik. Rad prikazuje motivaciju problema konsenzusa, osnovna svojstva algoritma, njegove glavne faze, arhitekturu simulatora, podržane scenarije i ograničenja implementacije.

Ključne reči: distribuirani sistemi, konsenzus, Byzantine Fault Tolerance, HotStuff, sertifikat kvoruma, replikovana mašina stanja.

Sadržaj

1	Uvod i motivacija	1
2	Model sistema i osnovni pojmovi	1
2.1	Replike, greške i kvorum	1
2.2	Pogledi, vođa i stablo predloga	2
2.3	Glasovi i sertifikati kvoruma	2
3	Basic HotStuff algoritam	3
3.1	Faza NEW-VIEW	3
3.2	Faza PREPARE i bezbednosni predikat	3
3.3	Faza PRE-COMMIT	3
3.4	Faza COMMIT i zaključavanje	4
3.5	Faza DECIDE i izvršavanje	4
3.6	Promena pogleda i živost	4
4	Sigurnost i živost	4
5	Arhitektura implementiranog simulatora	5
5.1	Domenski model i replika	5
5.2	Simulirana mreža	6
5.3	Kriptografski sloj	6
5.4	Pacemaker	6
5.5	Replikovana bankarska mašina stanja	6

5.6	Orkestracija, logovanje i veb prikaz	7
6	Simulacioni scenariji	7
7	Provera implementacije i ograničenja	8
8	Zaključak	8

1 Uvod i motivacija

Distribuirani sistem se sastoji od više procesa ili računara koji zajednički pružaju jednu uslugu. Njegova pouzdanost ne zavisi samo od ispravnosti pojedinačnih komponenti, već i od toga da li sve ispravne komponente imaju saglasan pogled na redosled događaja. Ako bi dve replike bankarskog sistema primenile iste transakcije različitim redosledom, njihova konačna stanja mogla bi da se razlikuju. Zbog toga je potrebno da replike postignu *konsenzus*: dogovor o jednoj vrednosti, odluci ili uređenom nizu komandi.

Posebno težak slučaj predstavljaju vizantijske greške. Neispravna replika tada nije ograničena na pad procesa ili kašnjenje, već može proizvoljno menjati ponašanje, slati kontradiktorne poruke različitim učesnicima ili pokušati da glasa za više suprotstavljenih predloga. Ovaj problem je formalizovan kroz problem vizantijskih generala [2]. BFT (*Byzantine Fault Tolerant*) protokol zato mora da obezbedi dva osnovna svojstva:

- **sigurnost (safety)** – dve ispravne replike ne smeju odlučiti različite vrednosti;
- **živost (liveness)** – sistem treba da nastavi da donosi odluke kada komunikacija postane dovoljno stabilna i vođstvo preuzme ispravna replika.

HotStuff je leaderski BFT protokol projektovan za delimično sinhroni model. Njegov značaj je u jednostavnom mehanizmu zaključavanja, sertifikatima kvoruma i promeni vođe sa linearnom komunikacionom složenosti. Posle stabilizacije mreže ispravan vođa može da napreduje brzinom stvarnog mrežnog kašnjenja, što se naziva odzivnošću (*responsiveness*) [1].

Cilj projekta je da ključne mehanizme algoritma učini vidljivim i eksperimentalno proverljivim. Zbog toga je realizovan simulator Basic HotStuff-a sa četiri replike i više scenarija: normalan rad, pad vođe, vizantijska ekvivokacija i velika mrežna kašnjenja. Implementacija nije produkcion konsenzusni servis, već obrazovni model koji jasno odvađa protokol konsenzusa, simuliranu mrežu, aplikativnu mašinu stanja i sloj za vizualizaciju [3].

2 Model sistema i osnovni pojmovi

2.1 Replike, greške i kvorum

Neka je n ukupan broj replika, a f najveći broj vizantijski neispravnih replika. U standardnom BFT modelu bezbedan rad zahteva

$$n \geq 3f + 1. \quad (1)$$

Veličina kvoruma u projektu određena je kao

$$q = n - f. \quad (2)$$

Za minimalnu konfiguraciju $n = 3f + 1$ ovo je $q = 2f + 1$. Simulator koristi $n = 4$, $f = 1$ i $q = 3$. Dva kvoruma od po tri replike u skupu od četiri replike imaju najmanje dve zajedničke replike. Pošto najviše jedna može biti vizantijska, u preseku se nalazi najmanje jedna ispravna replika. U kombinaciji sa pravilom da ispravna replika glasa najviše jednom po fazi i pogledu, ovaj presek sprečava formiranje dva suprotstavljena sertifikata kvoruma.

Komunikacija je autentifikovana: simulirani transport utvrđuje identitet pošiljaoca, a glasovi su digitalno potpisani. Mreža može kasniti, odbacivati poruke ili označiti repliku kao srušenu. Delimična sinhronost znači da nije unapred poznato kada će mreža postati stabilna, ali se pretpostavlja da posle nekog trenutka poruke između ispravnih replika stižu u konačno ograničenom vremenu.

2.2 Pogledi, vođa i stablo predloga

Izvršavanje je podeljeno na numerisane poglede (*views*). Svaki pogled ima jednog deterministički izabranog vođu. U implementaciji se koristi kružni raspored:

$$\text{leader}(v) = \text{Node}_{((v-1) \bmod n)+1} . \quad (3)$$

Tako je **Node-1** vođa pogleda 1, **Node-2** pogleda 2 itd. Pad ili neaktivnost vođe ne narušava sigurnost, ali privremeno zaustavlja napredak, pa replike posle isteka tajmera prelaze u novi pogled.

Predlozi su čvorovi stabla. Svaki čvor sadrži heš roditelja, celu klijentsku komandu, sopstveni heš i pogled u kome je predložen. Heš se računa nad roditeljem i kompletnom komandom, čime se kriptografski vezuju sadržaj i položaj čvora u istoriji. Poseban koren stabla označen je vrednošću **GENESIS**.

2.3 Glasovi i sertifikati kvoruma

Replika glasa potpisivanjem uređene trojke

$$\langle \text{phase}, \text{view}, \text{nodeHash} \rangle . \quad (4)$$

Kada vođa prikupi q validnih glasova različitih replika za istu trojku, objedinjuje ih u sertifikat kvoruma (*Quorum Certificate*, QC). QC predstavlja sažet dokaz da je potreban broj replika podržao određeni čvor u konkretnoj fazi i pogledu.

Rad o HotStuff-u pretpostavlja pragovnu šemu potpisa sa operacijama sličnim **tsign**, **tcombine** i **tverify**. Simulator taj interfejs idealizuje: svaka replika koristi stvarni Ed25519 par ključeva, svaki potpis se proverava, a objedinjeni sertifikat nosi neproziran SHA-256 token povezan sa proverenim udelima potpisa koji se čuvaju u memoriji. Time se verno simuliraju identitet, vezivanje glasa za poruku, jedinstvenost glasova i validacija QC-a, ali se ne implementira produkciona distribuirana pragovna šema kao što je BLS.

3 Basic HotStuff algoritam

Implementirana je neulančana varijanta Basic HotStuff-a iz Algoritma 2 originalnog rada [1]. Svaka odluka prolazi kroz eksplicitne faze prikazane na slici 1. Projekat ne implementira Chained HotStuff niti preklapa faze više uzastopnih predloga.



Slika 1: Faze jedne instance Basic HotStuff protokola.

3.1 Faza NEW-VIEW

Na početku ili pri prelasku u novi pogled svaka replika šalje novom vođi poruku **NEW-VIEW**. Poruka nosi najviši poznati **prepareQC**, ako takav sertifikat postoji. Vođa čeka poruke od $n - f$ jedinstvenih replika i među validnim sertifikatima bira onaj sa najvećim brojem pogleda, označen kao **highQC**. Na ovaj način novi vođa saznaje koja grana ima najjače prethodno opravdanje i ne započinje proizvoljnu istoriju.

Kada ima dovoljan broj **NEW-VIEW** poruka i jednu neizvršenu klijentsku komandu, vođa pravi čvor čiji je roditelj čvor iz **highQC**. Ako još ne postoji QC, roditelj je **GENESIS**. Zatim emituje **PREPARE** predlog svim replikama.

3.2 Faza PREPARE i bezbednosni predikat

Replika prihvata predlog samo ako ga je poslao vođa tekućeg pogleda, ako je heš čvora ispravan, ako predlog produžava čvor naveden u opravdavajućem QC-u i ako je sam QC validan. Pored toga proverava se centralni HotStuff predikat **safeNode**. Neka je b predloženi čvor, qc njegovo opravdanje, a $lockedQC$ trenutna brava replike. Predlog je bezbedan ako važi

$$\text{extends}(b, \text{lockedQC}.node) \quad \vee \quad qc.view > \text{lockedQC}.view. \quad (5)$$

Prvi uslov čuva postojeću bezbednu granu, dok drugi dozvoljava otključavanje kada noviji QC predstavlja jači dokaz o napretku sistema. Ako replika još nema bravu, svaki strukturno i kriptografski validan predlog smatra se bezbednim. Replika zatim šalje vođi jedan **PREPARE** glas.

3.3 Faza PRE-COMMIT

Vođa od q validnih **PREPARE** glasova za isti čvor formira **prepareQC** i emituje poruku **PRE-COMMIT**. Replike proveravaju da se tip, pogled i heš čvora poklapaju sa očekivanom

fazom, pamte sertifikat kao svoj najviši **PrepareQC** i šalju **PRE-COMMIT** glas vodi. Sačuvani **PrepareQC** biće uključen u eventualnu kasniju promenu pogleda.

3.4 Faza COMMIT i zaključavanje

Kada prikupi kvorum **PRE-COMMIT** glasova, vođa stvara **precommitQC** i emituje poruku **COMMIT**. Ispravna replika, nakon verifikacije, postavlja

$$lockedQC \leftarrow precommitQC \quad (6)$$

i šalje **COMMIT** glas. Zaključavanje je ključni lokalni mehanizam sigurnosti: replika ubuduće neće glasati za granu koja je u konfliktu sa zaključanom, osim ako predlog nosi opravdanje iz strogo novijeg pogleda, u skladu sa jednačinom (5).

3.5 Faza DECIDE i izvršavanje

Od q **COMMIT** glasova vođa formira **commitQC** i emituje **DECIDE**. Replika proverava sertifikat, rekonstruiše putanju od poslednjeg izvršenog čvora do odlučenog čvora i izvršava sve ranije neizvršene komande redosledom roditelj–dete. Identifikatori izvršenih komandi se pamte, pa se komanda ne primenjuje dva puta. Nakon uspešne odluke replika prelazi u sledeći pogled i tajmer se vraća na osnovnu vrednost.

3.6 Promena pogleda i živost

Svaka replika poseduje pacemaker sa tajmerom za tekući pogled. Ako odluka ne bude doneta pre isteka, pacemaker generiše prekid **nextView**, replika povećava broj pogleda i šalje **NEW-VIEW** sledećem vodi. Posle neuspešnog pogleda vreme čekanja se udvostručuje do granice od 32 osnovna intervala. Eksponecijalno odlaganje omogućava da, nakon stabilizacije mreže, sistem konačno dobije dovoljno dug pogled sa ispravnim vođom. Buduće poruke se privremeno baferuju, a zastarele odbacuju.

4 Sigurnost i živost

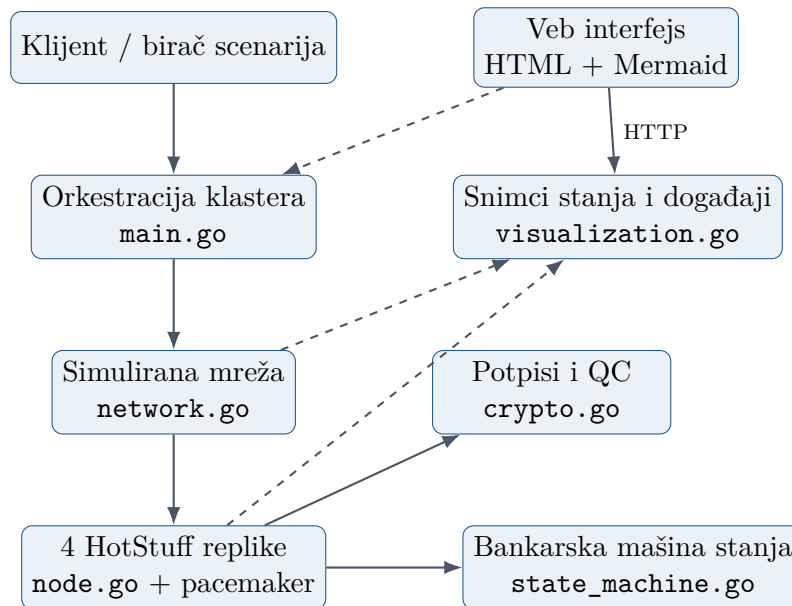
Sigurnost implementacije oslanja se na kombinaciju preseka kvoruma, jedinstvenog glasa i zaključavanja. Pretpostavimo da bi u istom pogledu postojala dva konfliktna QC-a iste faze. Njihovi skupovi potpisnika morali bi da dele najmanje jednu ispravnu repliku, ali ona ne može potpisati dva različita heša u istoj fazi i pogledu. Zbog toga takva dva QC-a ne mogu biti formirana. Kroz različite poglede, pravilo **safeNode** osigurava da ispravne replike ostanu na zaključanoj grani ili prihvate samo predlog opravdan strogo novijim QC-em.

Implementacija dodatno proverava identitet pošiljaoca, članstvo replike, integritet heša čvora, tip i pogled QC-a, jedinstvenost glasova i vezu predloga sa opravdavajućim roditeljem. **DECIDE** bez validnog **commitQC** ne može pokrenuti aplikativno izvršavanje. Ove provere su značajne jer sama struktura poruke nije dokaz njene autentičnosti.

Živost nije garantovana tokom proizvoljno dugog perioda asinhronosti ili dok se stalno biraju neispravni vođe. Međutim, u delimično sinhronom modelu kružna rotacija vođe i rastući tajmer dovode do pogleda u kome je vođa ispravan i vreme čekanja veće od efektivnog mrežnog kašnjenja. Tada ispravne replike mogu prikupiti potreban kvorum i završiti sve faze.

5 Arhitektura implementiranog simulatora

Simulator je napisan u jeziku Go. Sve replike rade unutar jednog procesa kao konkurentne gorutine i komuniciraju kanalima. Ovakva arhitektura olakšava ponovljivo pokretanje scenarija i pregled unutrašnjeg stanja, ali ne predstavlja fizički distribuirano izvršavanje. Glavni slojevi prikazani su na slici 2.



Slika 2: Slojevi i glavne veze u simulatoru. Pune linije predstavljaju izvršni tok, a isprekidane linije tok podataka za prikaz.

5.1 Domenski model i replika

Datoteka `domain.go` definiše zajedničke tipove: klijentsku komandu, tip protokolske poruke, čvor stabla, udeo potpisa, QC i kompletnu poruku. Centralna struktura `Node` u `node.go` sadrži trenutno stanje pogleda i faze, **PrepareQC**, **LockedQC**, lokalno stablo čvorova, red neobrađenih komandi, evidenciju izvršenih komandi, već datih glasova, prikupljenih glasova i poruka iz budućih pogleda.

Svaka replika ima tri ulazna toka: protokolske poruke, klijentske komande i obaveštenja o isteku tajmera. Glavna petlja bira događaj koji je spreman i serijski menja lokalno stanje pod zaštitom mutex-a. Time se protokolska logika jedne replike održava determinističnom, dok replike međusobno rade konkurentno.

5.2 Simulirana mreža

`network.go` registruje kanale replika i pruža operacije slanja, emitovanja i distribucije klijentskih komandi. Transport prepisuje polje identiteta stvarnim identitetom pošiljaoca, tako da učesnik ne može jednostavno da se predstavi kao druga replika. Mogu se podesiti slučajno kašnjenje, stopa odbacivanja poruka i pad čvora. Slanje se izvršava asinhrono u gorutini.

Mreža sadrži i zajednički `NodeStore`. Replika preko njega može da dohvati nedostajući čvor ili njegove pretke kada proverava produženje grane. To pojednostavljuje obrazovni simulator; u realnom sistemu takav sadržaj bi se pribavljao posebnim mrežnim protokolom i čuvao u lokalnom trajnom skladištu.

5.3 Kriptografski sloj

`crypto.go` pri formiranju klastera generiše Ed25519 ključeve za sve replike. Funkcija za potpisivanje vezuje fazu, pogled i heš čvora. Verifikator odbacuje nepoznate potpisnike, nevalidne potpise, duplikate i udele koji ne pripadaju istoj trojci. Tek kada postoji najmanje tri jedinstvena validna udela, kreira se token sertifikata. QC je validan samo ako token upućuje na zapamćeni skup udela sa identičnim tipom, pogledom i hešom.

5.4 Pacemaker

`pacemaker.go` radi kao zasebna gorutina po replici. Prima zahteve za aktiviranje tajmera, pamti aktivni pogled i po isteku obaveštava pripadajući čvor. Neuspeh udvostručuje interval, dok uspešan `DECIDE` vraća osnovni interval. Ovaj modul je odvojen od bezbednosnih pravila konsenzusa: on određuje kada treba pokušati sledeći pogled, ali ne određuje koji predlog replika sme da podrži.

5.5 Replikovana bankarska mašina stanja

`state_machine.go` predstavlja aplikativni sloj. Svaka replika počinje sa istim saldima: Marko 1000, Ana 200 i Luka 100. Podržane su tri determinističke komande:

Važno je razlikovati odluku konsenzusa od rezultata aplikativne validacije. Komanda može biti bezbedno uređena i izvršena na svim replikama, a ipak odbijena pravilima banke. U osnovnom scenariju prvo se blokira Markov račun, zatim se uređuje pokušaj prenosa 500 jedinica sa tog računa ka Ani, koji aplikacija odbija, i na kraju se Ani odobrava kredit

Tabela 1: Komande bankarske mašine stanja.

Komanda	Efekat
TRANSFER	Prenosi sredstva ako pošiljalac nije blokiran, iznos je validan i saldo je dovoljan.
BLOCK_ACCOUNT	Označava nalog kao blokiran.
APPROVE_LOAN	Evidentira kredit i uvećava saldo korisnika, pod uslovom da zahtev nije duplikat i da nalog nije blokiran.

od 2000 jedinica. Pošto sve replike polaze iz istog stanja i primenjuju isti uređeni niz determinističkih pravila, završavaju u identičnom stanju.

5.6 Orkestracija, logovanje i veb prikaz

`main.go` formira klaster, izračunava f i kvorum, pokreće replike i bira scenario. Program se može koristiti iz komandne linije ili preko HTTP servera na portu 8081. Krajnje tačke `/run`, `/logs` i `/visual-state` pokreću scenario i izlažu samo podatke potrebne za prikaz.

`logger.go` čuva niti-bezbedan tekstualni dnevnik. `visualization.go` pravi konzistentne snimke replika, blokova, QC-ova i transportnih događaja. Veb stranica na svakih 350 ms preuzima stanje i poslednje poruke, pretvara ih u Mermaid `sequenceDiagram` i prikazuje SVG zajedno sa dnevnikom. Vizualizacioni sloj ne menja stanje protokola, pa se može posmatrati kao read-only projekcija simulatora.

6 Simulacioni scenariji

Projekat obuhvata pet scenarija koji se mogu pokrenuti i koji su navedeni u tabeli 2. Njihova svrha nije merenje produkcionih performansi, već demonstracija konkretnih protokolskih svojstava.

Tabela 2: Podržani scenariji simulatora.

Scenario	Ponašanje koje se posmatra
Happy Path	Četiri aktivne replike uređuju i izvršavaju tri bankarske komande bez greške vođe.
Banking Commands	Naglašava razliku između konsenzusnog uređivanja i aplikativnog prihvatanja komande.
Silent Leader	Node-1, vođa pogleda 1, pada; ostale replike menjaju pogled i napreduju sa novim vođom.
Byzantine Equivocation	Prvi vođa šalje konfliktne predloge različitim replikama i uskraćuje svoj glas; pravilo jednog glasa sprečava dva konfliktna QC-a.
Delayed Network	Poruke kasne do tri sekunde; eksponencijalno produžavanje tajmera omogućava kasniji stabilan pogled.

Posebno je ilustrativan scenario ekvivokacije. Zlonamerni vođa šalje jedan predlog **Node-2**, a suprotstavljeni predlog replikama **Node-3** i **Node-4**, pri čemu namerno ne daje sopstveni glas. Nijedan konfliktan skup ne može da prikupi tri potpisa. Posle uklanjanja neispravnog vođe, sledeći ispravan vođa predlaže bezbednu komandu i sistem nastavlja rad. Scenario tako neposredno prikazuje vezu između veličine kvoruma i otpornosti na jednu vizantijsku repliku.

7 Provera implementacije i ograničenja

Repozitorijum sadrži testove za integritet heša kompletne komande, odbacivanje udela potpisa iz različitih pogleda, pravilo jednog glasa po fazi i pogledu, odbacivanje falsifikovane **DECIDE** poruke, jednako izvršavanje u normalnom scenariju i konzistentnost **JSON** stanja za vizualizaciju [3]. Testovi se pokreću komandama `go test ./...` i `go test -race ./...`, dok `go vet ./...` obavlja statičku proveru.

Glavna ograničenja simulatora su sledeća:

- replike su gorutine u jednom procesu, a ne zasebni mrežni čvorovi;
- autentifikacija transporta i pragovni **QC** predstavljaju kontrolisane simulacione apstrakcije;
- zajednički **NodeStore** pojednostavljuje pribavljanje nedostajućih predaka;
- nema trajnog skladišta, oporavka posle restarta, klijentskih potvrda niti zaštite od ponovljenih mrežnih zahteva;
- konfiguracija i korisnički interfejs su prvenstveno prilagođeni za četiri replike;
- implementiran je Basic HotStuff, bez ulančavanja, protočne obrade i optimizacija produkcionih sistema.

Ova ograničenja ne umanjuju nastavnu vrednost projekta. Naprotiv, uklanjanjem infrastrukturnih detalja u prvom planu ostaju poruke, **QC**-ovi, promena pogleda, zaključavanje i izvršavanje replikovane mašine stanja. Ipak, rezultati simulatora ne mogu se neposredno tumačiti kao pokazatelji performansi ili bezbednosti produkcione BFT platforme.

8 Zaključak

U projektu je realizovan funkcionalan, vizuelno pristupačan simulator Basic HotStuff BFT algoritma. Sistem sa četiri replike demonstrira kako kvorum od tri potpisa, autentifikovane poruke, pravilo jednog glasa i **safeNode** predikat zajedno sprečavaju donošenje konfliktan odluka u prisustvu jedne vizantijske replike. Eksplicitne faze **NEW-VIEW**, **PREPARE**, **PRE-COMMIT**, **COMMIT** i **DECIDE** olakšavaju praćenje trenutka u kome nastaju sertifikati i

brava, dok pacemaker pokazuje kako se napredak obnavlja posle pada vođe ili velikih kašnjenja.

Povezivanje protokola sa bankarskom mašinom stanja pokazuje praktičnu svrhu konsenzusa: algoritam ne odlučuje da li je poslovna komanda ispravna, već svim ispravnim replikama obezbeđuje identičan redosled u kome će se deterministička pravila primeniti. Veb interfejs i živa sekvenca poruka dodatno povezuju teorijske pojmove sa stvarnim tokom izvršavanja.

Literatura

- [1] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta i I. Abraham, “HotStuff: BFT Consensus with Linearity and Responsiveness”, *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC '19)*, str. 347–356, 2019. DOI: [10.1145/3293611.3331591](https://doi.org/10.1145/3293611.3331591).
- [2] L. Lamport, R. Shostak i M. Pease, “The Byzantine Generals Problem”, *ACM Transactions on Programming Languages and Systems*, vol. 4, br. 3, str. 382–401, 1982. DOI: [10.1145/357172.357176](https://doi.org/10.1145/357172.357176).
- [3] O. Perić, “Basic HotStuff BFT Simulator”, izvorni kod i projektna dokumentacija, GitHub repozitorijum, verzija pregledana na commitu 5e87f097d8dc51cf1cf504d12a515eb39bf13bf8, 15. jul 2026. <https://github.com/ognjen217/HotStuffBFT>.